

ADVANCED OPERATING SYSTEMS

Lab 3 — Locking and lockless reads

Vittorio Zaccaria

Politecnico di Milano

From a race to synchronization

What this lab compares

The lab uses two deliberately small experiments:

1. a scalar lost update, repaired with an atomic read-modify-write;
2. one shared-list workload implemented as **unsafe**, **spinlock**, and **RCU** variants.

The list experiment separates three questions:

- **Correctness:** can a reader observe a partially updated invariant?
- **Serialization:** must readers exclude one another and the writer?
- **Lifetime:** when may an updater reclaim an object previously visible to a reader?

RCU means lockless read-side traversal here. It does **not** mean no synchronization: writers still serialize publication and defer reclamation.

Use more than one CPU

A one-vCPU guest can interleave threads but cannot execute them simultaneously. Use either frontend from Lab 1 and boot QEMU with four virtual CPUs.

With the host-terminal frontend:

```
# Host
T=amd64 E=full make dev-up
T=amd64 E=full make dev-build
T=amd64 E=full SMP=4 make dev-run
```

With the browser frontend, open the module source from the Explorer, use **Terminal → Run Build Task**, then run the `AOS: Run QEMU (4 vCPUs)` task. Editing, clangd feedback, the build log, and the QEMU guest console remain in the same browser window.

Inside the guest:

```
grep -c '^processor' /proc/cpuinfo
```

The expected count is 4. All reported timings are from instrumented QEMU and are useful only for relative comparisons in the same run.

Atomic read-modify-write

The lost-update window

The racy worker performs a load, a computation, and a store:

```
long value = shared_value;
cpu_relax();                    /* widen the window between load and store */
shared_value = value + delta;
```

Each load and each store is a single instruction, but the sequence is not: two CPUs can load the same old value and one overwrites the other’s result. Marking the accesses with `READ_ONCE` / `WRITE_ONCE` or `volatile` would not help, because they constrain how the compiler emits each access, not which CPU owns the update.

One atomic operation

The fixed worker replaces the three-step sequence with one atomic read-modify-write:

```
atomic64_add(delta, &atomic_value);
```

The two workers start together, execute the same number of `+1` and `-1` operations, and complete before module initialization returns. The expected final value is zero.

Atomic counter source

Run the two variants

```
# Guest, four vCPUs
insmod /modules/lab-3-th-atomics.ko variant=0
rmmod module
insmod /modules/lab-3-th-atomics.ko variant=1
rmmod module
```

Observed in one verified run:

```
counter_demo variant=racy   cpus=4 result=1078378 expected=0
counter_demo variant=atomic cpus=4 result=0       expected=0
```

The exact racy result varies and could occasionally be zero by coincidence. The atomic invariant is the meaningful result. Do not compare the elapsed times of these two implementations: `cpu_relax()` intentionally widens the race window.

What atomics do not solve

An atomic operation protects one scalar update. It does not automatically protect:

- a relationship among several fields;
- a multi-step list modification;
- the lifetime of an object reached through a pointer;
- arbitrary accesses to unrelated memory.

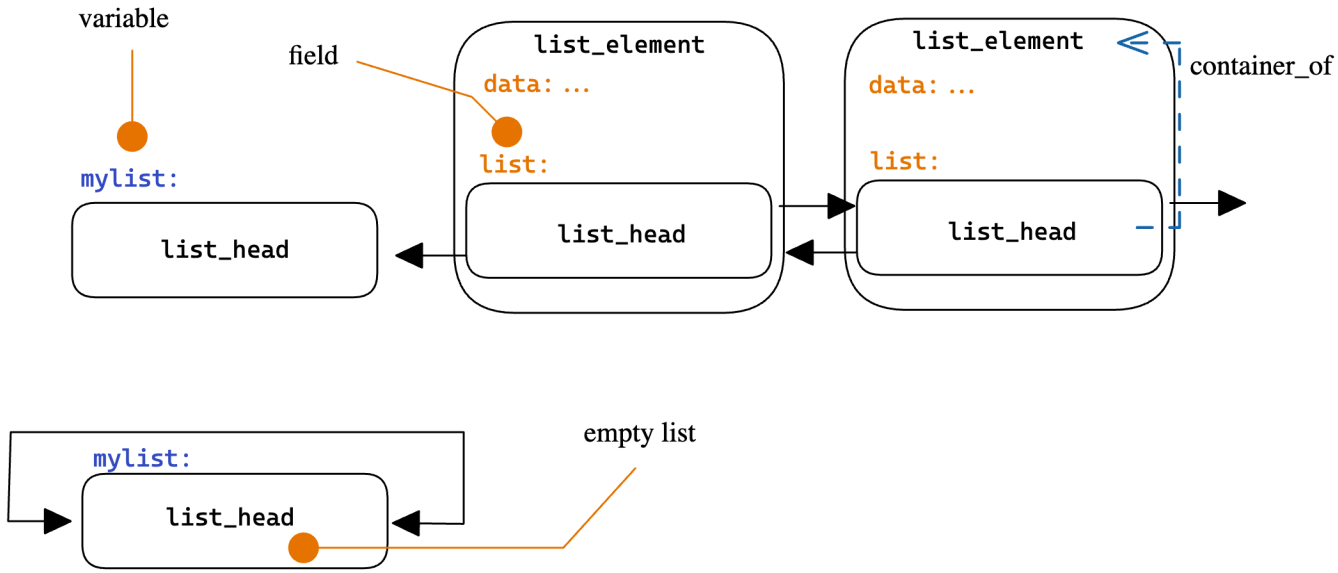
The list experiment needs a stronger synchronization contract.

One shared-list workload

Embedded kernel lists

Linux embeds `struct list_head` inside each object:

```
struct benchmark_node {
  u64 value;
  u64 check;
  struct list_head link;
  struct rcu_head rcu;
};
```



“Each object owns the links that connect it to the shared list.”

The list contains 64 nodes by default. Every node must satisfy the compound invariant:

```
node->check == ~node->value
```

Readers traverse the entire list and count invariant violations. One writer repeatedly increments the first node. The three variants keep the same observable contract but change synchronization.

Common benchmark controls

```
variant=0|1|2          unsafe | spinlock | RCU
readers=1..8           reader kthreads
duration_ms=3000       measurement interval
writer_pause_us=1000   pause between updates
list_nodes=64          list length
```

Reader counters are private to each thread and summed only after all workers finish. `printk` is outside the timed region. Module initialization waits for the bounded experiment, prints one result line, and returns with no worker left running.

Variant 0 — unsafe

A torn compound update

The unsafe writer exposes two separate stores:

```
value = node->value + 1;
node->value = value;
udelay(1);                    /* widen the observation window */
node->check = ~value;
```

Readers use no lock. They can observe the new `value` with the old `check`: the two stores are separate, and nothing prevents a reader from running between them.

This is safer and more repeatable than intentionally corrupting list links or freeing an object under a reader: the demonstration reports a violated invariant without crashing the teaching VM.

Observe the violations

```
insmod /modules/lab-3-th-rcu.ko \
  variant=0 readers=3 duration_ms=1000 writer_pause_us=1000
rmmod module
```

Verified example:

```
variant=unsafe cpus=4 readers=3 duration_ms=1000
traversals=2908844 updates=748 violations=25402
```

The counts vary, but `violations` should normally be non-zero on the four-vCPU setup. A zero count would not prove correctness; races are defined by missing synchronization, not by one observed failure.

Variant 1 — spinlock

Serialize readers and writer

The lock-based reader holds the same spinlock for its entire traversal:

```
spin_lock(&list_lock);
list_for_each_entry(node, &shared_list, link)
  check(node);
spin_unlock(&list_lock);
```

The writer holds it across both field updates:

```
spin_lock(&list_lock);
node->value = value;
udelay(1);
node->check = ~value;
spin_unlock(&list_lock);
```

The invariant is now protected, but every reader excludes every other reader and the writer. Do not sleep while holding a spinlock.

Correctness and scaling

Run one reader, then three:

```
insmod /modules/lab-3-th-rcu.ko \
  variant=1 readers=1 duration_ms=1000 writer_pause_us=1000
rmmod module
insmod /modules/lab-3-th-rcu.ko \
  variant=1 readers=3 duration_ms=1000 writer_pause_us=1000
rmmod module
```

Verified examples:

```
spinlock readers=1 traversals=941737 updates=753 violations=0
spinlock readers=3 traversals=285492 updates=671 violations=0
```

Adding readers reduced aggregate traversals because they contend for one lock. These are QEMU observations, not universal performance constants.

Variant 2 — RCU

Read-side critical section

RCU readers take no lock:

```
rcu_read_lock();
list_for_each_entry_rcu(node, &shared_list, link)
  check(node);
rcu_read_unlock();
```

The read-side critical section promises that an object reached through the RCU-protected list will not be reclaimed until the reader exits. Readers still need an immutable or otherwise safely synchronized view of each object.

Copy, publish, reclaim

There is exactly one writer, so no lock is needed on the update side either: RCU only requires that updaters do not overlap, and here nothing else modifies the list while the benchmark runs. With several writers, a lock among writers would still be required.

The writer prepares a complete replacement, then publishes it with one structural change:

```
replacement = kmalloc(sizeof(*replacement), GFP_KERNEL);
/* initialize replacement */

old = list_first_entry(&shared_list, struct benchmark_node, link);
list_replace_rcu(&old->link, &replacement->link);

kfree_rcu(old, rcu);
```

`list_replace_rcu` is five pointer assignments (`include/linux/rculist.h`):

```
new->next = old->next;
new->prev = old->prev;
rcu_assign_pointer(list_next_rcu(new->prev), new);   /* publication */
```

```
new->next->prev = new;
old->prev = LIST_POISON2;
```

The new node is wired to its neighbours while still private. `rcu_assign_pointer` stores `prev->next = new` with release semantics, so any reader that follows the new pointer also sees the initialized fields; `list_for_each_entry_rcu` pairs with it through `rcu_dereference`. Readers move only forward, so the back pointer is a plain store. The old node keeps a valid `next`, so a reader still standing on it continues normally; this is why it cannot be freed immediately.

`kfree_rcu` delays reclamation until pre-existing readers have crossed a grace period. Module initialization calls `rcu_barrier()` after the benchmark so all queued reclamation callbacks finish before teardown proceeds.

Correctness and scaling

```
insmod /modules/lab-3-th-rcu.ko \
  variant=2 readers=1 duration_ms=1000 writer_pause_us=1000
rmmod module
insmod /modules/lab-3-th-rcu.ko \
  variant=2 readers=3 duration_ms=1000 writer_pause_us=1000
rmmod module
```

Verified examples:

```
RCU readers=1 traversals=895864  updates=803 violations=0
RCU readers=3 traversals=2538763 updates=797 violations=0
```

With three readers, aggregate RCU traversal throughput scaled while the spinlock variant serialized. The writer kept its update rate too, because readers no longer hold it back; each update still pays for an allocation and a deferred free.

Interpret the comparison

Same machine, relative result

One verified one-second run on four QEMU vCPUs produced:

VARIANT	READERS	TRAVERSALS	UPDATES	VIOLATIONS
unsafe	3	2,908,844	748	25,402
spinlock	1	941,737	753	0
spinlock	3	285,492	671	0
RCU	1	895,864	803	0
RCU	3	2,538,763	797	0

Interpret the direction, not the exact numbers:

- unsafe is fast but incorrect;
- spinlock is correct and simple, but readers serialize;
- RCU is correct and lets read-side throughput scale;
- RCU moves complexity and cost to the update and reclamation side.

Optional write-heavy run

Set `writer_pause_us=0` to remove the deliberate read-mostly pause:

```
insmod /modules/lab-3-th-rcu.ko \
  variant=1 readers=3 duration_ms=1000 writer_pause_us=0
rmmod module
insmod /modules/lab-3-th-rcu.ko \
  variant=2 readers=3 duration_ms=1000 writer_pause_us=0
rmmod module
```

This changes the workload rather than producing a general ranking. RCU is designed for read-mostly structures; the relative result depends on list size, update method, allocation, grace periods, vCPU scheduling and host load.

Safe lifecycle

Threads belong to the module

Both modules follow the same lifecycle:

1. validate parameters and allocate state;
2. create every kthread and bind it to an available vCPU;
3. release a common start gate;
4. wait for every worker completion;
5. report after all workers have exited;
6. reclaim pending RCU callbacks before returning;
7. free the current list on module removal.

A module must never return from cleanup while one of its kthreads can still execute module code. Creation failures must unwind already-created tasks and allocations.

End the lab

Check that no module remains loaded:

```
cat /proc/modules
```

Exit QEMU with `Ctrl-]`, then `x`. Stop the container for the frontend you used:

```
# Host-terminal frontend
T=amd64 E=full make dev-down

# Browser frontend
make vscode-down
```

The lesson is not “RCU is always faster.” It is: choose synchronization from the invariant, access pattern and lifetime contract, then measure the actual workload.